

Integración de la placa Teensy 4.1 con la cámara de visión artificial OpenMV Cam H7 Plus para robótica con inteligencia artificial embebida

Conexión entre ambas placas, funcionamiento del reconocimiento de imágenes con Edge Impulse, y aplicación a un robot capaz de seguir una línea, detectar objetos y jugar al fútbol de forma autónoma.

Alcance: especificaciones de hardware, protocolo de comunicación, flujo de trabajo de machine learning embebido y arquitectura de control combinada.

Fuentes: documentación oficial de PJRC (Teensy), OpenMV y Edge Impulse, foros técnicos y bibliografía de RoboCupJunior Soccer.

Índice

- 1** Objetivo y alcance del proyecto
 - 2** La placa Teensy 4.1
 - 3** La cámara OpenMV Cam H7 Plus
 - 4** Edge Impulse y el reconocimiento de imágenes
 - 5** Conexión física entre la Teensy y la cámara
 - 6** Protocolo de comunicación (biblioteca RPC)
 - 7** Seguidor de línea
 - 8** Detector de objetos
 - 9** Robot que juega al fútbol
 - 10** Arquitectura conjunta con las dos placas
 - 11** Lista de materiales
 - 12** Preguntas y puntos a verificar antes de empezar
-

Objetivo y alcance del proyecto

El objetivo de este trabajo es investigar y documentar la integración de dos placas de hardware con propósitos complementarios: la **Teensy 4.1**, un microcontrolador de alto rendimiento orientado al control en tiempo real, y la **OpenMV Cam H7 Plus**, una cámara con capacidad de procesamiento de imágenes e inteligencia artificial embebida mediante la plataforma **Edge Impulse**.

Se abordan cuatro problemas concretos:

- Cómo conectar físicamente ambas placas y qué protocolo utilizar para intercambiar datos entre ellas.
- Cómo funciona el reconocimiento de imágenes con Edge Impulse sobre la cámara, desde la captura de datos hasta el modelo ejecutándose en el dispositivo.
- Cómo implementar, con las dos placas ya conectadas, un robot seguidor de línea y un detector de objetos.
- Cómo extender esa misma arquitectura a un robot que juegue al fútbol de forma autónoma, detectando una pelota y un arco.

NOTA SOBRE EL ENFOQUE

En cada uno de los cuatro problemas existe más de una solución técnica válida. Este informe explica los enfoques recomendados por los propios fabricantes y usados en proyectos documentados, y señala en la sección 12 las decisiones de diseño que quedan abiertas y conviene resolver antes de empezar a construir.

La placa Teensy 4.1

La Teensy 4.1 es una placa de desarrollo fabricada por PJRC, basada en el microcontrolador NXP iMXRT1062, con núcleo ARM Cortex-M7 funcionando a 600 MHz. Es compatible con el entorno Arduino IDE mediante el complemento Teensyduino, lo que permite programarla con las mismas funciones y bibliotecas del ecosistema Arduino, con un rendimiento muy superior al de placas convencionales de 8 bits.

2.1 Especificaciones relevantes para este proyecto

Teensy 4.1 — NXP iMXRT1062 (ARM Cortex-M7, 600 MHz)

- 1024 KB RAM (512 KB de acceso ultrarrápido TCM)
- 8 MB de memoria flash
- 55 pines digitales · 18 entradas analógicas
- 8 puertos serie UART (Serial1 a Serial8) disponibles como periféricos independientes
- Alimentación lógica: 3.3 V · pines tolerantes a 5 V en la entrada
- Ethernet 10/100 integrado
- Puerto host USB + micro-USB de programación
- Ranura para tarjeta microSD

Por qué se elige para este proyecto: sobra potencia de cómputo para lógica de control, PID, gestión de motores y comunicación simultánea, dejando el procesamiento de imagen

2.2 Características que condicionan el diseño del robot

Característica	Valor	Relevancia para el proyecto
Frecuencia de reloj	600 MHz (permite overclock)	Suficiente para correr un lazo de control PID a alta frecuencia mientras se atiende la comunicación serie
Puertos serie (UART)	8 independientes	Permite dedicar un puerto exclusivo a la cámara sin interferir con otros periféricos
Salidas PWM	35 pines	Necesarias para controlar la velocidad de los motores mediante un driver tipo puente H
Nivel lógico	3.3 V	Coincide con el nivel lógico de la OpenMV Cam H7 Plus; no se necesita convertor de niveles entre ambas
Alimentación	USB o VIN externo	La corriente de los motores debe alimentarse aparte, nunca desde el pin de 3.3 V ni 5V de la Teensy

PUNTO CRÍTICO DE DISEÑO ELÉCTRICO

Los motores DC nunca deben alimentarse directamente desde los pines de la Teensy. Se requiere un driver de motores (ver sección 10) con su propia fuente de alimentación, y la Teensy solo entrega las señales de control lógico al driver.

La cámara OpenMV Cam H7 Plus

La OpenMV Cam H7 Plus es una placa de visión artificial de bajo consumo pensada para ejecutar algoritmos de procesamiento de imagen directamente sobre el hardware, sin depender de una computadora externa ni de conexión a internet. Se programa en MicroPython a través del entorno OpenMV IDE, que además permite visualizar en vivo lo que la cámara está capturando.

3.1 Especificaciones técnicas

OpenMV Cam H7 Plus — STM32H743II (ARM Cortex-M7, 480 MHz)

- Sensor de imagen OV5640, 5 megapíxeles
- Resolución máxima: 2592 × 1944 px
- 32 MB de SDRAM externa + 1 MB de SRAM interna
- 32 MB de flash externa (almacena los modelos entrenados)
- Programación en MicroPython, vía OpenMV IDE
- Interfaces: UART, I2C, SPI, CAN
- Conector USB (puerto virtual COM + unidad de almacenamiento)
- Conector para batería LiPo de 3.7 V

Es la única variante de la familia OpenMV con memoria suficiente para ejecutar redes neuronales convolucionales completas entrenadas con Edge Impulse; los modelos FOMO de detección de objetos requieren esta versión "Plus".

3.2 Funciones de visión que trae incorporadas de fábrica

Antes de entrenar cualquier modelo propio con Edge Impulse, conviene saber que la cámara ya incluye, en su firmware base, varias funciones de visión clásica (sin inteligencia artificial) que resuelven buena parte de lo pedido en el proyecto sin necesidad de entrenar nada:

Función incorporada	Aplicación directa
Seguimiento de blobs por color (<code>find_blobs</code>)	Seguidor de línea y detección de objetos de colores conocidos (pelota, arcos)
Detección de rostros	No aplica directamente a este proyecto, mencionado por completitud
AprilTags y códigos QR	Alternativa para identificar objetos con marcadores en vez de reconocimiento visual
Detección de líneas y rectángulos	Complemento al seguimiento por color en el seguidor de línea

DISTINCIÓN IMPORTANTE

El seguimiento por color (`find_blobs`) no utiliza Edge Impulse ni redes neuronales: es un algoritmo de procesamiento de imagen clásico, incluido de fábrica, y es la técnica que efectivamente usan la mayoría de los robots de RoboCupJunior Soccer para detectar la pelota y el arco. Edge Impulse se vuelve necesario cuando el objeto a reconocer no se puede aislar de forma confiable por color (ver sección 4).

Edge Impulse y el reconocimiento de imágenes

Edge Impulse es una plataforma en la nube para entrenar modelos de aprendizaje automático destinados a ejecutarse en dispositivos con recursos limitados (microcontroladores), disciplina conocida como **TinyML**. La OpenMV Cam H7 Plus tiene integración oficial con Edge Impulse desde 2020, lo que permite entrenar un modelo sin escribir código de red neuronal y exportarlo directamente en un formato que la cámara puede ejecutar.

4.1 Flujo de trabajo completo



Los pasos 1 a 4 ocurren en la nube (studio.edgeimpulse.com); el paso 5 vuelca el modelo entrenado a la memoria flash de la cámara.

En la práctica, la captura de imágenes se realiza conectando la cámara a una computadora por USB y usando el propio OpenMV IDE, o con la herramienta de línea de comandos `edge-impulse-uploader --format-openmv`, que sube un directorio de imágenes ya organizado por carpetas (una carpeta por clase) directamente al proyecto de Edge Impulse.

4.2 FOMO: el algoritmo que hace posible detectar objetos en un microcontrolador

El obstáculo histórico para hacer detección de objetos (no solo clasificación) en un microcontrolador es que los modelos clásicos, como MobileNet SSD, necesitan calcular las coordenadas de un cuadro delimitador (*bounding box*) por cada objeto, lo que exige mucha memoria RAM y potencia de cómputo, generalmente fuera del alcance de una placa embebida.

Edge Impulse resuelve esto con **FOMO** (*Faster Objects, More Objects*), un algoritmo propio que renuncia a calcular cuadros delimitadores y en cambio predice únicamente el **centroide** (punto central) de cada objeto detectado, junto con su clase. Esta simplificación reduce el uso de memoria RAM por debajo de los 200 KB y hace que el modelo corra hasta 30 veces más rápido que MobileNet SSD, a costa de no informar el tamaño del objeto.

Aspecto	Clasificación de imagen	FOMO (detección con centroide)	Detección clásica (bounding box)
Qué informa	una sola etiqueta por imagen completa	posición y clase de cada objeto	posición, tamaño y clase de cada objeto
Cuántos objetos por imagen	uno (el dominante)	varios	varios
Memoria RAM requerida	baja	menos de 200 KB	varios MB
Corre en OpenMV H7 Plus	sí	sí	sí, con más limitaciones de velocidad

LIMITACIÓN DE FOMO A TENER EN CUENTA

Integración Teensy 4.1 · OpenMV Cam H7 Plus · Edge Impulse

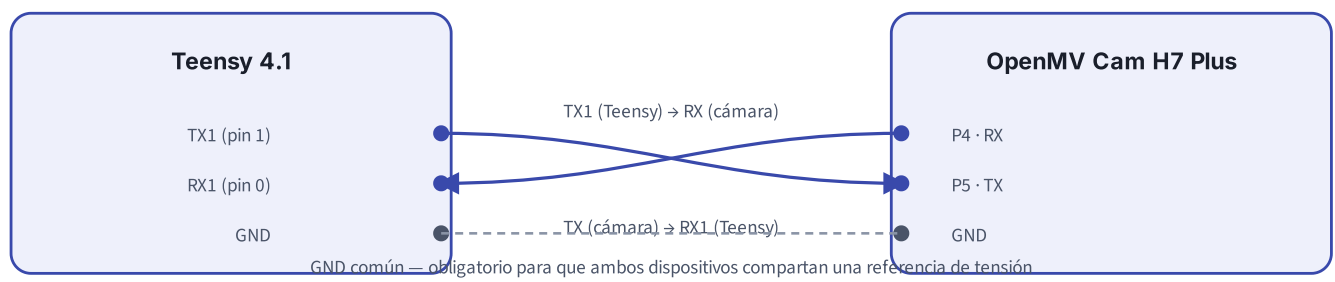
FOMO funciona mejor cuando los objetos de una misma clase tienen tamaño similar entre sí y no están muy próximos unos de otros en la imagen. Para una pelota de un tamaño conocido y un arco, esta

5

Conexión física entre la Teensy y la cámara

La forma recomendada de conectar ambas placas para este proyecto es mediante un puerto serie **UART** (comunicación asíncrona punto a punto). Es la interfaz más simple de las cuatro que soporta la OpenMV Cam H7 Plus (UART, I2C, SPI, CAN) y es la que utiliza la biblioteca oficial de comunicación entre OpenMV y Arduino/Teensy.

5.1 Esquema de conexión



Los cables se cruzan: el TX de una placa siempre va al RX de la otra. Conectar TX con TX (o RX con RX) es el error más común al hacer esta conexión y provoca que ninguna de las dos placas reciba datos.

5.2 Por qué UART y no otra interfaz

Interfaz	Velocidad en la OpenMV H7 Plus	Cables necesarios	Adecuada para este proyecto
UART	hasta 7.5 Mb/s	2 datos + GND	Recomendada
I2C	hasta 1 Mb/s	2 datos + GND	Válida, más lenta
SPI	hasta 80 Mb/s	4 datos + GND	Más rápida pero más cableado y configuración
CAN	hasta 1 Mb/s	2 datos + GND	Pensada para redes con varios nodos, no necesaria aquí

Con un solo canal de comunicación entre dos placas, la velocidad de UART es más que suficiente: lo que se transmite en este proyecto no es la imagen completa, sino resultados ya procesados (coordenadas de un objeto, un valor de error, un código de estado), que ocupan apenas unos pocos bytes por mensaje.

5.3 Compatibilidad eléctrica

Ambas placas trabajan a **3.3 V** de lógica, por lo que se conectan directamente sin necesidad de un convertor de niveles lógicos. Esto simplifica considerablemente el cableado respecto de combinar, por ejemplo, un Arduino Uno de 5 V con la cámara.

ALIMENTACIÓN DE LA CÁMARA

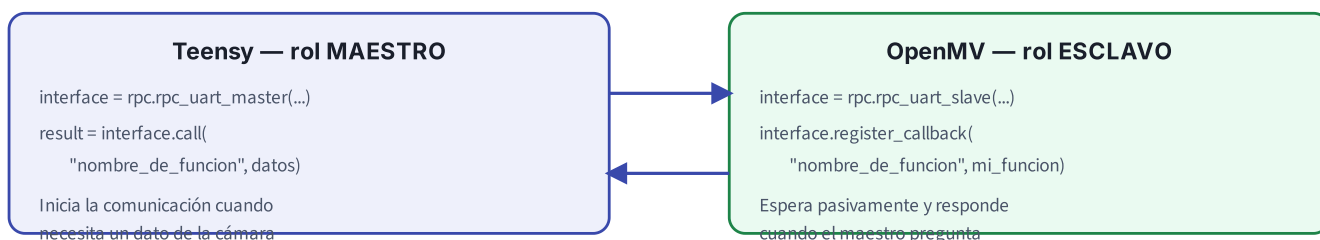
La OpenMV Cam H7 Plus puede alimentarse desde el propio cable USB de programación, desde una batería LiPo de 3.7 V conectada a su puerto dedicado, o desde los pines de 3.3 V/5V si la Teensy tiene capacidad de entregar esa corriente. Conviene decidir esto antes de fijar el diseño mecánico del robot, ya que condiciona dónde va la batería.

Protocolo de comunicación: la biblioteca RPC

OpenMV ofrece una biblioteca llamada **RPC** (*Remote Procedure Call*, llamada a procedimiento remoto) diseñada específicamente para conectar la cámara con un Arduino, un ESP32 o una Teensy. Existe en dos versiones equivalentes: una para MicroPython, incorporada de fábrica en la cámara, y una para Arduino/Teensyduino que se instala como biblioteca ([openmv-arduino-rpc](#) , disponible en el gestor de bibliotecas del IDE de Arduino).

6.1 Cómo funciona el modelo maestro/esclavo

El patrón de diseño de la biblioteca es maestro/esclavo. En la mayoría de los proyectos de este tipo, la **Teensy actúa como maestro** (decide cuándo pedir información y qué hacer con las respuestas) y la **cámara actúa como esclava** (ejecuta funciones de visión bajo demanda y devuelve resultados).



6.2 Qué resuelve esta biblioteca que una comunicación serie simple no resuelve

- **Empaquetado automático de datos.** No es necesario definir a mano un formato de mensaje: la biblioteca serializa y deserializa los datos (enteros, flotantes, cadenas de bytes) de forma transparente.
- **Tolerancia a fallos.** Se recupera automáticamente ante pérdida de paquetes, cables que se aflojan momentáneamente o desincronización entre ambos extremos, algo frecuente en un robot en movimiento con vibración.
- **Independencia de la interfaz física.** El mismo código de aplicación funciona sin cambios sobre UART, I2C, SPI o CAN; solo cambia la línea donde se crea el objeto de interfaz.

6.3 Alternativa más simple: UART directa sin RPC

Para casos donde solo se necesita enviar dos o tres números (por ejemplo, la posición X de un blob y su tamaño), es válido y más liviano prescindir de la biblioteca RPC y usar un protocolo propio muy simple sobre `pyb.UART` en la cámara y `Serial1` en la Teensy: por ejemplo, empaquetar los valores con la biblioteca `struct` de Python y leerlos del lado de la Teensy con `Serial1.readBytes()`. Varios proyectos documentados de robots de seguimiento y de fútbol (incluyendo publicaciones en el foro oficial de OpenMV) usan este enfoque directo en vez de RPC quien lo consideran innecesariamente complejo para intercambiar solo coordenadas.

RECOMENDACIÓN

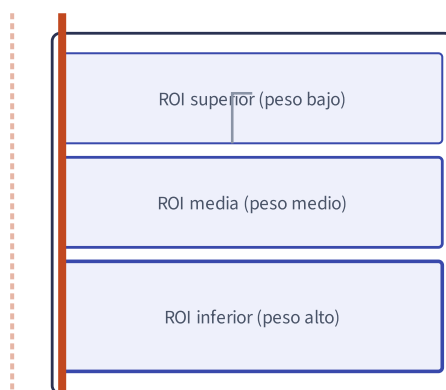
Para el seguidor de línea y el detector de objetos, alcanza con el protocolo simple. RPC se justifica si más adelante se necesita, además, controlar remotamente parámetros de la cámara desde la Teensy (cambiar de modo, ajustar umbrales de color, etc.) sin reprogramarla.

Seguidor de línea

El seguidor de línea es la aplicación más directa de la cámara en modo de seguimiento por color, sin necesidad de Edge Impulse. La OpenMV Cam H7 Plus procesa la imagen y calcula un valor de error; la Teensy recibe ese error y ajusta la velocidad de los dos motores para corregir el rumbo.

7.1 Detección de la línea en la cámara

El método estándar, documentado en los ejemplos oficiales de OpenMV, divide la imagen en varias **regiones de interés (ROI)** horizontales y busca en cada una el centro de masa de la línea mediante umbral de color o de escala de grises.



Cada ROI (región de interés) busca el centro de la línea por separado.

La región inferior (más cercana al robot) tiene más peso: es la más confiable para saber hacia dónde girar.

línea (ej. negra sobre piso blanco)

Las posiciones encontradas en cada ROI se promedian con distinto peso, dando más importancia a la región inferior (la más cercana al robot), y ese promedio ponderado es el **error**: la distancia entre el centro de la línea detectada y el centro de la imagen.

Seguidor de línea • control PID

7.2 Corrección con control PID en la Teensy

El error calculado por la cámara se envía por UART a la Teensy, que lo utiliza como entrada de un controlador PID (proporcional, integral, derivativo) para calcular cuánto corregir la velocidad de cada motor.

Término	Qué hace	Efecto de subirlo demasiado
Kp (proporcional)	corrige en proporción directa al error actual	oscilación en tramos rectos
Ki (integral)	corrige el error acumulado en el tiempo	sobrecorrección y oscilaciones lentas
Kd (derivativo)	anticipa la corrección según la velocidad de cambio del error	respuesta nerviosa, inestabilidad

La fórmula que aplica el controlador, ya trasladada a las dos ruedas, es:

$$\text{corrección} = K_p \cdot \text{error} + K_i \cdot (\text{suma de errores}) + K_d \cdot (\text{error} - \text{error_anterior})$$

$$\text{velocidad_motor_izquierdo} = \text{velocidad_base} + \text{corrección} \quad \cdot \quad \text{velocidad_motor_derecho} = \text{velocidad_base} - \text{corrección}$$

El ajuste de las constantes se hace de forma experimental: se empieza con $K_i = K_d = 0$ y se sube K_p hasta que el robot sigue la línea, aunque oscile. Después se introduce K_d para amortiguar esa oscilación, y por último, solo si queda un error sostenido en las curvas, se agrega una K_i pequeña.

CASO QUE HAY QUE PROGRAMAR APARTE

Cuando ninguna ROI encuentra la línea (por ejemplo, en una intersección o al perder la pista), el controlador PID no tiene información válida. Es necesario un estado separado de "búsqueda" — usualmente, seguir girando hacia el último lado donde se vio la línea— antes de retomar el PID normal.

Detector de objetos

A diferencia del seguidor de línea, un detector de objetos genérico (reconocer una pieza, una figura, un obstáculo específico que no se distingue por color) sí requiere entrenar un modelo propio con Edge Impulse, siguiendo el flujo descrito en la sección 4.

8.1 Pasos concretos con esta cámara

1. Crear una cuenta y un proyecto en studio.edgeimpulse.com.
2. Capturar imágenes de cada objeto a reconocer desde distintos ángulos, distancias e iluminaciones, usando el modo de adquisición de datos del OpenMV IDE conectado a la cámara.
3. Etiquetar las imágenes: para FOMO, marcar el centro aproximado de cada objeto y asignarle una clase.
4. En el diseño del impulso ("Create impulse"), agregar un bloque de procesamiento de imágenes y un bloque de aprendizaje de tipo **Object Detection**, seleccionando específicamente un modelo **FOMO**.
5. Entrenar el modelo (esto ocurre en los servidores de Edge Impulse, no en la cámara).
6. Validar la precisión con el conjunto de prueba que la plataforma separa automáticamente.
7. En la pestaña de despliegue ("Deployment"), elegir **OpenMV firmware** y compilar. Esto genera un archivo de firmware ya combinado con el modelo entrenado.
8. Cargar ese firmware a la cámara desde el OpenMV IDE (menú Herramientas → Run Bootloader) y ejecutar el script de ejemplo [ei_object_detection.py](#) que se incluye en la descarga.

8.2 Qué hace falta programar del lado de la cámara

El script generado por Edge Impulse ya realiza la inferencia (ejecutar el modelo sobre cada cuadro capturado) y expone, por cada detección, la clase, la posición del centroide en píxeles y el nivel de confianza. Lo que queda a cargo de quien programa el proyecto es decidir qué hacer con esos resultados: por ejemplo, enviar por UART hacia la Teensy únicamente la posición y la clase del objeto de mayor confianza detectado en el cuadro.

8.3 Dos formas de exportar el modelo

Opción	Cómo funciona	Cuándo conviene
OpenMV firmware	reemplaza el firmware completo de la cámara por uno que ya incluye el modelo	modelos que no entran en memoria como biblioteca; es el método que recomienda Edge Impulse
OpenMV library	copia el modelo (.tflite) y las etiquetas a la unidad de almacenamiento de la cámara, y se lo importa desde un script	solo para modelos de clasificación pequeños; puede dar problemas de memoria con FOMO

SOBRE LA CUANTIZACIÓN DEL MODELO

OpenMV solo admite modelos cuantizados en formato entero de 8 bits (int8). Si Edge Impulse ofrece por defecto un modelo en punto flotante, hay que seleccionar explícitamente la opción "Quantized (int8)" antes de compilar el firmware para OpenMV.

Robot que juega al fútbol

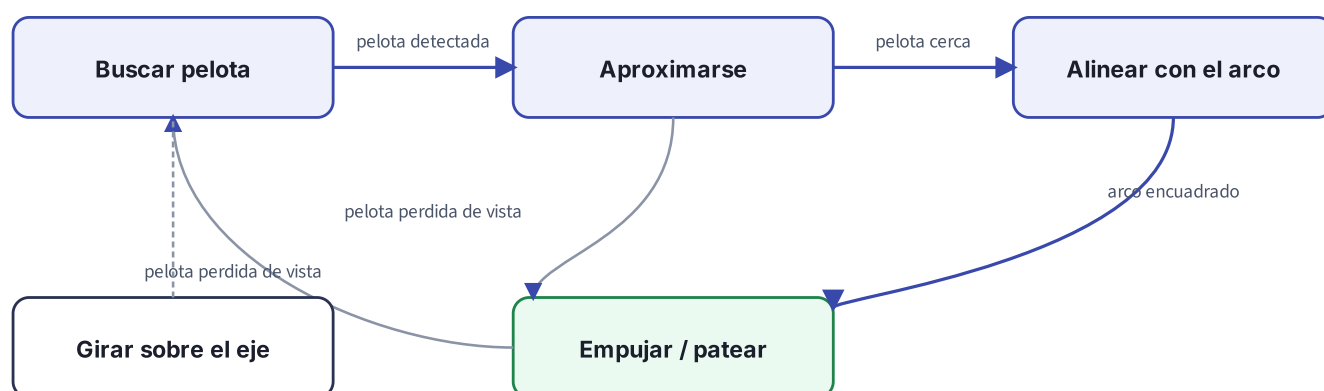
Este caso combina las dos técnicas anteriores: seguimiento por color para la pelota y el arco (siguiendo el enfoque estándar en RoboCupJunior Soccer, la competencia de referencia mundial en robótica escolar de este tipo) y, opcionalmente, un modelo de Edge Impulse si se necesita distinguir el arco propio del rival por una forma en vez de por color.

9.1 Por qué la detección por color es el estándar en esta disciplina

Las reglas de RoboCupJunior Soccer especifican que los robots deben detectar una pelota (normalmente naranja) y anotar en un arco identificado por color. Esta convención existe precisamente porque el seguimiento de blobs por color es liviano, rápido (decenas de cuadros por segundo) y no requiere entrenar ningún modelo, lo que resulta más apropiado para un sistema que debe reaccionar en tiempo real durante un partido.

9.2 Máquina de estados del comportamiento

La lógica de un robot de este tipo se organiza habitualmente como una máquina de estados finita, donde la cámara aporta en cada cuadro la posición de la pelota y del arco, y la Teensy decide en qué estado está y qué motor mover.



Estado inicial / de respaldo cuando no hay pelota a la vista

9.3 Qué envía la cámara y qué decide la Teensy en cada estado

Estado	Dato que necesita de la cámara	Acción de la Teensy
Girar sobre el eje	si hay o no un blob de color "pelota" en el cuadro	girar en el lugar hasta encontrarla
Buscar / aproximarse	posición X del centroide de la pelota y su tamaño (proxy de distancia)	orientar el robot hacia la pelota y avanzar
Alinear con el arco	posición X del centroide del arco	girar sobre el eje manteniendo la pelota cerca, hasta encuadrar el arco
Empujar / patear	confirmación de alineación	avanzar en línea recta o activar un mecanismo de pateo

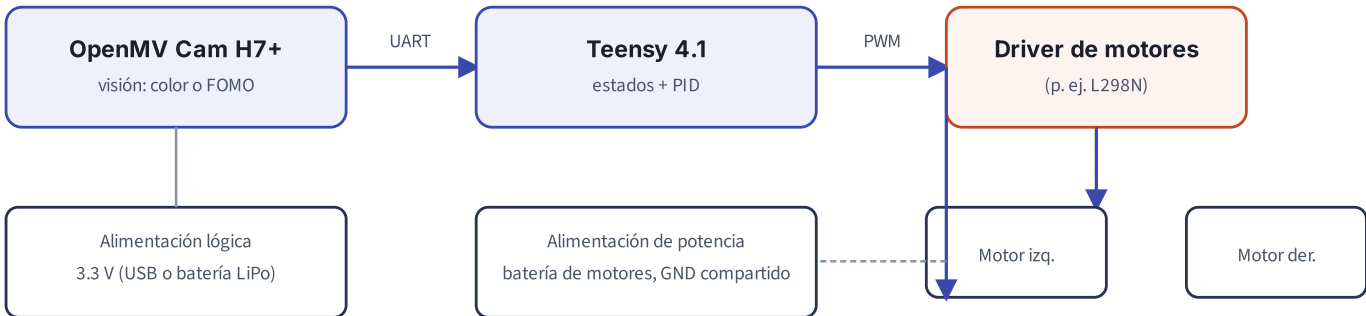
9.4 Repartición de tareas entre las dos placas

Integración Teensy 4.1 · OpenMV Cam H7 Plus · Edge Impulse

- **La cámara** resuelve exclusivamente percepción: en cada cuadro, entrega la posición y el tamaño de los blobs de color de la pelota y del arco (o, si se usa un modelo entrenado, la clase y posición

Arquitectura conjunta con las dos placas

Tanto el seguidor de línea como el detector de objetos y el robot de fútbol comparten la misma arquitectura general de sistema, cambiando únicamente qué procesa la cámara y qué decide la Teensy en cada caso.



Regla de diseño: la Teensy es el único punto de decisión; la cámara solo informa y el driver solo ejecuta. Esto facilita la depuración: cada bloque se puede probar por separado.

10.1 El eslabón que falta: del microcontrolador al movimiento

Ni la Teensy ni la OpenMV pueden alimentar motores DC directamente desde sus pines: la corriente que necesita un motor supera ampliamente lo que un pin de microcontrolador puede entregar. Se necesita un **driver de motores**, un circuito intermedio (típicamente basado en un puente H) que recibe señales de control de bajo voltaje desde la Teensy y conmuta la corriente de una fuente de alimentación separada hacia los motores.

Elemento	Rol	Conexión típica
Driver L298N (u equivalente)	controla dirección y velocidad de dos motores DC	4 pines de dirección (IN1-IN4) + 2 pines PWM (ENA, ENB) desde la Teensy
PWM (modulación por ancho de pulso)	regula la velocidad promedio entregada al motor	generado por los pines PWM de la Teensy hacia los pines de habilitación del driver
Fuente de potencia para motores	alimenta el driver y, a través de él, los motores	batería independiente; su tierra (GND) debe unirse a la tierra lógica de la Teensy

10.2 Resumen de responsabilidades por bloque

- **OpenMV Cam H7 Plus:** captura y procesa imágenes (por color o con un modelo de Edge Impulse), envía resultados numéricos por UART.
- **Teensy 4.1:** recibe esos resultados, ejecuta la lógica de control (PID o máquina de estados) y genera señales PWM y de dirección.
- **Driver de motores:** traduce esas señales en corriente real hacia los motores, usando una fuente de alimentación separada.

Lista de materiales

Componente	Cantidad orientativa	Función en el proyecto
Teensy 4.1	1	controlador principal, lógica de decisión y control de motores
OpenMV Cam H7 Plus	1	visión artificial: color o modelos de Edge Impulse (FOMO)
Driver de motores tipo puente H (ej. L298N)	1	controla dirección y velocidad de los motores DC
Motores DC con reducción (o motorreductores)	2	tracción del robot
Ruedas	2 (+ rueda loca)	desplazamiento
Batería para lógica (LiPo 3.7 V o similar)	1	alimenta Teensy y cámara si no van por USB
Batería para motores	1	alimenta el driver y los motores, separada de la lógica
Cables Dupont o similar	según diseño	conexión UART entre placas y señales al driver
Chasis / estructura mecánica	1	soporte físico de todos los componentes
Pelota naranja reglamentaria (si aplica RoboCupJunior)	1	objetivo de detección del robot de fútbol
Cinta o superficie de pista con línea de color contrastante	—	pista de prueba para el seguidor de línea

HERRAMIENTAS DE SOFTWARE NECESARIAS (SIN COSTO)

- Arduino IDE con el complemento Teensyduino, para programar la Teensy.
- OpenMV IDE, para programar la cámara y visualizar en vivo lo que detecta.
- Cuenta gratuita en studio.edgeimpulse.com, solo si se va a entrenar un modelo de detección de objetos propio.

Preguntas y puntos a verificar antes de empezar

Esta sección reúne decisiones de diseño y datos concretos que no quedan resueltos únicamente con la documentación general, y conviene confirmar antes de avanzar con la construcción.

¿El proyecto exige usar Edge Impulse, o alcanza con seguimiento por color?

Si el objeto a detectar (pelota, línea, pieza) tiene un color que se puede aislar de forma confiable en el ambiente donde va a funcionar el robot, el seguimiento por color es más simple, más rápido y no requiere entrenar nada. Conviene revisar la consigna del proyecto para saber si el uso de inteligencia artificial es un requisito explícito de la evaluación.

¿Qué tensión y corriente entregan los motores elegidos?

Esto determina qué driver de motores es compatible y qué batería de potencia hace falta. Es un dato que depende del motor concreto que se compre y no se puede fijar de forma genérica.

¿La cámara se alimentará por USB, batería propia, o desde la Teensy?

Condiciona el diseño mecánico (dónde va cada batería) y si hace falta un regulador de tensión adicional.

¿Dónde vive la lógica de decisión: en la Teensy o en la cámara?

Como se señaló en la sección 9.4, es una decisión de arquitectura que conviene fijar antes de escribir código en ambas placas.

¿Qué mecanismo de pateo o empuje usará el robot de fútbol?

Este informe cubre la detección y el acercamiento a la pelota, pero no aborda el mecanismo físico de pateo (un motor adicional, un solenoide, o simplemente empujar con la carcasa), que depende del diseño mecánico específico del robot y de si existe un reglamento de competencia a seguir.

¿Hay un reglamento de competencia que fije el tamaño del robot, el color exacto de la pelota o del arco, o el formato de la pista?

Si el proyecto sigue las reglas de RoboCupJunior Soccer u otra competencia similar, esos reglamentos fijan parámetros exactos (tamaño de cancha, tono de color permitido, tiempo de partido) que van a condicionar los umbrales de color y las distancias de detección a calibrar.

¿El detector de objetos necesita distinguir clases visualmente similares?

Si los objetos a reconocer se diferencian por forma o textura y no por color, hace falta entrenar un modelo con Edge Impulse (sección 8). Si se diferencian por color, alcanza con el seguimiento de blobs, sin entrenar nada.

¿Qué velocidad de imagen (FPS) necesita la aplicación?

El seguimiento por color en QVGA (320×240) corre habitualmente entre 25 y 50 cuadros por segundo en esta cámara, mientras que un modelo FOMO corre a una velocidad menor que depende del tamaño de imagen elegido al entrenarlo. Para un robot en movimiento rápido, esta diferencia puede ser determinante al elegir entre un método y otro.